

AWS Console-Only Setup Guide

Flipkart Clone Microservices Deployment

Complete single PDF: ECS Fargate, ECR, DocumentDB, ALB, Cloud Map, CloudWatch, WAF, CI/CD, Redis, CloudFront and monitoring.

Region: us-east-1 · Architecture: React + Node.js microservices + MongoDB-compatible DocumentDB

Prepared for the Flipkart Clone workspace project
Use this as a deployment lab manual and production checklist.

AWS Console-Only Setup Guide for Flipkart Microservices

Complete single PDF guide for deploying the Flipkart Clone microservices architecture on AWS.

Region used in this guide: **US East (N. Virginia)** us-east -1

Project services:

- api-gateway
- user-service
- product-service
- cart-service
- order-service
- payment-service
- notification-service
- frontend

Important note: Most infrastructure steps are AWS Console-only. However, pushing local Docker images to ECR requires Docker and either AWS CLI, GitHub Actions, CodeBuild, or another CI/CD builder. AWS Console alone cannot upload a Docker image directly from your laptop.

Table of Contents

1. AWS Account and Initial Setup
2. VPC and Network Setup
3. ECR Repository Setup
4. DocumentDB MongoDB-Compatible Database Setup
5. Application Load Balancer Setup
6. ECS Cluster Setup
7. ECS Task Definitions
8. Service Discovery with AWS Cloud Map
9. ECS Services Deployment
10. Auto Scaling Setup
11. CloudWatch Monitoring
12. Testing and Verification
13. Route 53 Custom Domain Optional
14. ACM SSL Certificate Optional
15. Cost Optimization
16. Backup and Disaster Recovery
17. Secrets Manager Setup
18. ElastiCache Redis Setup

19. CloudFront CDN Setup
20. AWS WAF Setup
21. AWS Shield Advanced Optional
22. AWS CodePipeline CI/CD
23. GitHub Actions CI/CD Alternative
24. Jenkins on EC2 Alternative
25. AWS X-Ray Tracing
26. Enhanced Container Insights
27. DataDog Integration Optional
28. Architecture Summary
29. Complete Setup Checklist
30. Cost Estimate
31. Troubleshooting
32. Next Steps

LAB 1: AWS Account and Initial Setup

Step 1.1: Sign into AWS Console

1. Open: `https://aws.amazon.com/console/`
2. Sign in with your AWS account.
3. Select region: **US East (N. Virginia)** `us-east-1`.

Step 1.2: Create IAM User for Development

1. Search for **IAM** in the AWS Console search bar.
2. Go to **Users** → **Add users**.
3. User name: `flipkart-admin`.
4. Select **AWS Management Console access**.
5. Choose **Custom password** and enter a strong password.
6. Uncheck **User must create a new password at next sign-in**.
7. Click **Next: Permissions**.
8. Choose **Attach existing policies directly**.
9. Search and select **AdministratorAccess**.
10. Click **Next: Tags** → **Next: Review** → **Create user**.
11. Save the login URL, username, and password securely.

Production best practice: Avoid long-term AdministratorAccess for daily use. Use least-privilege roles and MFA.

LAB 2: VPC and Network Setup

Step 2.1: Create VPC

1. Search for **VPC** in AWS Console.
2. Click **Create VPC**.
3. Select **VPC and more**.
4. Use this configuration:

Setting	Value
Name tag	flipkart-vpc
IPv4 CIDR block	10.0.0.0/16
Number of Availability Zones	2
Number of public subnets	2
Number of private subnets	2
NAT gateways	1 per AZ for production, None for cost-saving dev
VPC endpoints	None
DNS hostnames	Enabled
DNS resolution	Enabled

5. Click **Create VPC**.
6. Wait 2-3 minutes.
7. Note these values:
 - VPC ID
 - Public Subnet IDs
 - Private Subnet IDs

Step 2.2: Create Security Groups

Create the following security groups inside `flipkart-vpc`.

ALB Security Group

Name: `flipkart-alb-sg`

Inbound rules:

Type	Port	Source	Description
HTTP	80	0.0.0.0/0	Allow HTTP from internet
HTTPS	443	0.0.0.0/0	Allow HTTPS from internet

Outbound: Leave default **All traffic**.

API Gateway Security Group

Name: `flipkart-api-gateway-sg`

Inbound rules:

Type	Port	Source	Description
Custom TCP	3000	<code>flipkart-alb-sg</code>	Allow traffic from ALB

Outbound: Leave default **All traffic**.

Microservices Security Group

Name: `flipkart-services-sg`

Inbound rules:

Service	Port	Source
User Service	3001	<code>flipkart-api-gateway-sg</code>
Product Service	3002	<code>flipkart-api-gateway-sg</code>
Cart Service	3003	<code>flipkart-api-gateway-sg</code>
Order Service	3004	<code>flipkart-api-gateway-sg</code>
Payment Service	3005	<code>flipkart-api-gateway-sg</code>
Notification Service	3006	<code>flipkart-api-gateway-sg</code>

Recommended additional internal traffic rule:

Type	Port	Source	Description
All TCP	0-65535	<code>flipkart-services-sg</code>	Allow service-to-service traffic

DocumentDB Security Group

Name: `flipkart-docdb-sg`

Inbound rules:

Type	Port	Source	Description
Custom TCP	27017	<code>flipkart-services-sg</code>	Allow MongoDB from services

If API Gateway also connects directly to services through Cloud Map, ensure outbound rules remain open and service security groups allow required traffic.

LAB 3: ECR Repository Setup

Step 3.1: Create ECR Repositories

1. Search for **ECR**.
2. Click **Create repository**.
3. Create one private repository for each service:

- flipkart-api-gateway
- flipkart-user-service
- flipkart-product-service
- flipkart-cart-service
- flipkart-order-service
- flipkart-payment-service
- flipkart-notification-service
- flipkart-frontend

For each repository:

Setting	Value
Visibility	Private
Tag immutability	Disabled
Scan on push	Enabled
Encryption	AES-256

Step 3.2: Save Repository URIs

After creation:

1. Go to **ECR** → **Repositories**.
2. Open each repository.
3. Copy the URI.
4. Save all URIs in a notes file.

URI format:

```
ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/flipkart-api-gateway
```

Step 3.3: Push Docker Images to ECR

Console-only limitation: AWS Console can create repositories, but cannot build and push Docker images from your laptop. Use one of these:

- Local Docker + AWS CLI
- AWS CodeBuild

- GitHub Actions
- Jenkins

Local Docker + AWS CLI Example

Configure AWS CLI:

```
aws configure
# AWS Access Key ID: your key
# AWS Secret Access Key: your secret
# Default region: us-east-1
# Default output format: json
```

Login to ECR:

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin
ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com
```

Build and push API Gateway:

```
cd flipkart-clone/api-gateway
docker build -t flipkart-api-gateway .
docker tag flipkart-api-gateway:latest ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/flipkart-api-
gateway:latest
docker push ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/flipkart-api-gateway:latest
```

Repeat for:

```
user-service
product-service
cart-service
order-service
payment-service
notification-service
frontend
```


LAB 4: DocumentDB MongoDB-Compatible Database Setup

Step 4.1: Create DocumentDB Subnet Group

1. Search for **DocumentDB**.
2. Open **Subnet groups**.
3. Click **Create**.
4. Configure:

Setting	Value
Name	flipkart-docdb-subnet-group
Description	Subnet group for Flipkart DocumentDB
VPC	flipkart-vpc
Availability Zones	Select 2 AZs
Subnets	Select 2 private subnets

5. Click **Create**.

Step 4.2: Create DocumentDB Cluster

1. Go to **DocumentDB** → **Clusters**.
2. Click **Create**.
3. Configure:

Setting	Value
Cluster identifier	flipkart-docdb-cluster
Engine version	Latest available 5.x
Instance class	db.t3.medium
Number of instances	2 production, 1 cost-saving dev
Master username	docdbadmin
Master password	Strong password
VPC	flipkart-vpc
Subnet group	flipkart-docdb-subnet-group
VPC security group	flipkart-docdb-sg
Backup retention	7 days
Encryption at rest	Enabled
CloudWatch logs	Audit and profiler logs enabled

4. Click **Create cluster**.
5. Wait 10-15 minutes.

Step 4.3: Save Connection Details

Open the cluster and save:

- Cluster endpoint
- Reader endpoint
- Port: 27017
- Username and password

Connection string template:

```
mongodb://docdbadmin:PASSWORD@CLUSTER_ENDPOINT:27017/DATABASE_NAME?replicaSet=rs0&readPreference=secondaryPreferred&retryWrites=false
```

Examples:

```
mongodb://docdbadmin:PASSWORD@CLUSTER_ENDPOINT:27017/flipkart-  
users?replicaSet=rs0&readPreference=secondaryPreferred&retryWrites=false  
mongodb://docdbadmin:PASSWORD@CLUSTER_ENDPOINT:27017/flipkart-  
products?replicaSet=rs0&readPreference=secondaryPreferred&retryWrites=false
```

LAB 5: Application Load Balancer Setup

Step 5.1: Create Target Groups

Go to **EC2** → **Target Groups** → **Create target group**.

API Gateway Target Group

Setting	Value
Target type	IP addresses
Target group name	flipkart-api-tg
Protocol	HTTP
Port	3000
VPC	flipkart-vpc
Health check path	/health
Healthy threshold	2
Unhealthy threshold	3
Timeout	5 seconds
Interval	30 seconds
Success codes	200

Do not register targets manually. ECS will register them.

Frontend Target Group

Setting	Value
Target type	IP addresses
Target group name	flipkart-frontend-tg
Protocol	HTTP
Port	80
VPC	flipkart-vpc
Health check path	/healthz recommended, / acceptable
Success codes	200

Step 5.2: Create Application Load Balancer

1. Go to **EC2** → **Load Balancers**.
2. Click **Create Load Balancer**.
3. Select **Application Load Balancer**.

Setting	Value
Name	flipkart-alb
Scheme	Internet-facing
IP address type	IPv4

Setting	Value
VPC	flipkart-vpc
Mappings	Select 2 public subnets
Security group	flipkart-alb-sg
Listener	HTTP:80
Default action	Forward to flipkart-frontend-tg

4. Click **Create load balancer**.

Step 5.3: Add Listener Rule for API

1. Open flipkart-alb.
2. Go to **Listeners**.
3. Open HTTP:80 listener rules.
4. Add rule:

Field	Value
Condition	Path
Path pattern	/api/*
Action	Forward to flipkart-api-tg
Priority	1

5. Save.
6. Copy the ALB DNS name.

LAB 6: ECS Cluster Setup

Step 6.1: Create ECS Cluster

1. Search for **ECS**.
2. Go to **Clusters** → **Create Cluster**.
3. Configure:

Setting	Value
Cluster name	flipkart-cluster
Infrastructure	AWS Fargate serverless
Container Insights	Enabled

4. Click **Create**.

LAB 7: ECS Task Definitions

Create one task definition per service.

Common settings:

Setting	Value
Launch type	AWS Fargate
OS/Architecture	Linux/X86_64
Task execution role	ecsTaskExecutionRole
Log driver	awslogs
Region	us-east-1

Step 7.1: API Gateway Task Definition

Setting	Value
Family	flipkart-api-gateway
CPU	0.5 vCPU
Memory	1 GB
Container name	api-gateway
Image URI	ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/flipkart-api-gateway:latest
Container port	3000

Environment variables:

```
PORT=3000
JWT_SECRET=your-super-secret-jwt-key
USER_SERVICE_URL=http://user-service.flipkart.local:3001
PRODUCT_SERVICE_URL=http://product-service.flipkart.local:3002
CART_SERVICE_URL=http://cart-service.flipkart.local:3003
ORDER_SERVICE_URL=http://order-service.flipkart.local:3004
PAYMENT_SERVICE_URL=http://payment-service.flipkart.local:3005
NOTIFICATION_SERVICE_URL=http://notification-service.flipkart.local:3006
```

Logging:

```
awslogs-group=/ecs/flipkart-api-gateway
awslogs-region=us-east-1
awslogs-stream-prefix=ecs
```

Step 7.2: User Service Task Definition

Setting	Value
Family	flipkart-user-service
CPU	0.5 vCPU
Memory	1 GB
Container name	user-service
Image URI	ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/flipkart-user-service:latest
Container port	3001

Environment variables:

```
PORT=3001
JWT_SECRET=your-super-secret-jwt-key
MONGODB_URI=mongodb://docdbadmin:PASSWORD@CLUSTER_ENDPOINT:27017/flipkart-
users?replicaSet=rs0&readPreference=secondaryPreferred&retryWrites=false
```

Step 7.3: Product Service Task Definition

```
Family=flipkart-product-service
Container name=product-service
Image=ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/flipkart-product-service:latest
Port=3002
CPU=0.5 vCPU
Memory=1 GB
PORT=3002
MONGODB_URI=mongodb://docdbadmin:PASSWORD@CLUSTER_ENDPOINT:27017/flipkart-
products?replicaSet=rs0&readPreference=secondaryPreferred&retryWrites=false
```

Step 7.4: Cart Service Task Definition

```
Family=flipkart-cart-service
Container name=cart-service
Image=ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/flipkart-cart-service:latest
Port=3003
PORT=3003
MONGODB_URI=mongodb://docdbadmin:PASSWORD@CLUSTER_ENDPOINT:27017/flipkart-
carts?replicaSet=rs0&readPreference=secondaryPreferred&retryWrites=false
PRODUCT_SERVICE_URL=http://product-service.flipkart.local:3002
```

Step 7.5: Order Service Task Definition

```
Family=flipkart-order-service
Container name=order-service
Image=ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/flipkart-order-service:latest
Port=3004
PORT=3004
MONGODB_URI=mongodb://docdbadmin:PASSWORD@CLUSTER_ENDPOINT:27017/flipkart-
orders?replicaSet=rs0&readPreference=secondaryPreferred&retryWrites=false
CART_SERVICE_URL=http://cart-service.flipkart.local:3003
NOTIFICATION_SERVICE_URL=http://notification-service.flipkart.local:3006
```

Step 7.6: Payment Service Task Definition

```
Family=flipkart-payment-service
Container name=payment-service
Image=ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/flipkart-payment-service:latest
Port=3005
PORT=3005
MONGODB_URI=mongodb://docdbadmin:PASSWORD@CLUSTER_ENDPOINT:27017/flipkart-
payments?replicaSet=rs0&readPreference=secondaryPreferred&retryWrites=false
ORDER_SERVICE_URL=http://order-service.flipkart.local:3004
NOTIFICATION_SERVICE_URL=http://notification-service.flipkart.local:3006
```

Step 7.7: Notification Service Task Definition

```
Family=flipkart-notification-service
Container name=notification-service
Image=ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/flipkart-notification-service:latest
Port=3006
PORT=3006
MONGODB_URI=mongodb://docdbadmin:PASSWORD@CLUSTER_ENDPOINT:27017/flipkart-
notifications?replicaSet=rs0&readPreference=secondaryPreferred&retryWrites=false
SMTP_HOST=smtp.gmail.com
```

```
SMTP_PORT=587
SMTP_USER=your-email@example.com
SMTP_PASS=your-app-password
EMAIL_FROM=Flipkart Clone <no-reply@example.com>
```

Step 7.8: Frontend Task Definition

Setting	Value
Family	flipkart-frontend
CPU	0.25 vCPU
Memory	0.5 GB
Container name	frontend
Image URI	ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/flipkart-frontend:latest
Container port	80

Important: React environment variables are baked at build time. If your frontend Docker image was built with `/api`, no runtime env is required because the frontend Nginx can proxy `/api` to gateway in Docker. For ECS behind ALB, build frontend using:

```
REACT_APP_API_URL=/api
```

or rebuild with:

```
REACT_APP_API_URL=http://ALB_DNS_NAME/api
```


LAB 8: Service Discovery with AWS Cloud Map

Step 8.1: Create Namespace

1. Search for **Cloud Map**.
2. Click **Create namespace**.
3. Configure:

Setting	Value
Namespace name	flipkart.local
Namespace type	Private DNS namespace
VPC	flipkart-vpc

4. Click **Create namespace**.

Use Private DNS namespace for ECS service discovery inside the VPC. Then services can resolve names such as user-service.flipkart.local.

LAB 9: ECS Services Deployment

Step 9.1: Create Backend Services

Create services in this order:

1. user-service
2. product-service
3. notification-service
4. cart-service
5. order-service
6. payment-service
7. api-gateway-service
8. frontend-service

For each internal backend service:

Setting	Value
Cluster	flipkart-cluster
Compute options	Launch type
Launch type	FARGATE
Application type	Service
Desired tasks	2 production, 1 dev
Subnets	Private subnets
Public IP	Disabled
Security group	flipkart-services-sg
Load balancer	None
Service discovery	Enabled
Namespace	flipkart.local
DNS record type	A
TTL	60

Service discovery names:

ECS Service	Discovery name
user-service	user-service
product-service	product-service
cart-service	cart-service
order-service	order-service
payment-service	payment-service
notification-service	notification-service

Step 9.2: Create API Gateway Service

Setting	Value
Service name	api-gateway-service
Task definition	flipkart-api-gateway:LATEST
Desired tasks	2
Subnets	Private subnets
Security group	flipkart-api-gateway-sg
Load balancer type	Application Load Balancer
Load balancer	flipkart-alb
Listener	Existing HTTP:80
Target group	flipkart-api-tg
Service discovery name	api-gateway

Step 9.3: Create Frontend Service

Setting	Value
Service name	frontend-service
Task definition	flipkart-frontend:LATEST
Desired tasks	2
Subnets	Private subnets
Security group	Service SG that allows ALB to port 80
Load balancer type	Application Load Balancer
Load balancer	flipkart-alb
Target group	flipkart-frontend-tg

Recommended frontend security group:

Type	Port	Source
HTTP	80	flipkart-alb-sg

LAB 10: Auto Scaling Setup

Step 10.1: Configure ECS Service Auto Scaling

For each service:

1. Go to **ECS** → **Clusters** → `flipkart-cluster`.
2. Select the service.
3. Open **Auto Scaling**.
4. Configure:

Setting	Value
Min tasks	2
Desired tasks	2
Max tasks	10

CPU target tracking policy:

Setting	Value
Policy name	<code>cpu-scaling-policy</code>
Policy type	Target tracking
Metric	<code>ECSServiceAverageCPUUtilization</code>
Target value	70
Scale-out cooldown	60 seconds
Scale-in cooldown	60 seconds

Memory target tracking policy:

Setting	Value
Policy name	<code>memory-scaling-policy</code>
Metric	<code>ECSServiceAverageMemoryUtilization</code>
Target value	80

Apply to:

- `api-gateway-service`
- `user-service`
- `product-service`
- `cart-service`
- `order-service`
- `payment-service`
- `notification-service`
- `frontend-service`

LAB 11: CloudWatch Monitoring

Step 11.1: Create Dashboard

1. Search for **CloudWatch**.
2. Go to **Dashboards** → **Create dashboard**.
3. Name: Flipkart-Monitoring.

Step 11.2: Add Widgets

Add these widgets:

Widget	Metrics
ECS Service CPU	CPUUtilization for all services
ECS Service Memory	MemoryUtilization for all services
ALB Metrics	ActiveConnectionCount, RequestCount, TargetResponseTime
Target Health	HealthyHostCount, UnHealthyHostCount
DocumentDB	CPUUtilization, DatabaseConnections, ReadThroughput, WriteThroughput

Step 11.3: Create Alarms

Create alarms for:

Alarm	Threshold
API Gateway High CPU	CPU > 80%
Service High Memory	Memory > 90%
ALB Unhealthy Targets	UnHealthyHostCount > 0
DocumentDB High CPU	CPU > 80%
DocumentDB Connections	Near max connections

Create SNS topic:

flipkart-alerts

Subscribe your email and confirm subscription.

LAB 12: Testing and Verification

Step 12.1: Get Application URL

1. Go to **EC2** → **Load Balancers**.
2. Select `flipkart-alb`.
3. Copy **DNS name**.
4. Open:

`http://ALB_DNS_NAME`

Step 12.2: Test Endpoints

Health checks:

```
http://ALB_DNS_NAME/api/health
http://ALB_DNS_NAME/api/users/health
http://ALB_DNS_NAME/api/products/health
```

Register user:

```
curl -X POST http://ALB_DNS_NAME/api/users/register \
-H "Content-Type: application/json" \
-d '{
  "name": "Test User",
  "email": "test@example.com",
  "password": "password123",
  "phone": "1234567890"
}'
```

Login:

```
curl -X POST http://ALB_DNS_NAME/api/users/login \
-H "Content-Type: application/json" \
-d '{
  "email": "test@example.com",
  "password": "password123"
}'
```

Products:

```
curl http://ALB_DNS_NAME/api/products
```

Important: In the completed project, product route is `/api/products`, not `/api/products/products`.

Step 12.3: Verify ECS Services

1. Go to **ECS** → **Clusters** → `flipkart-cluster`.
2. Open each service.
3. Verify tasks are **RUNNING**.
4. Open task logs and check for errors.

Step 12.4: Verify DocumentDB Connection

Use an EC2 bastion instance in the same VPC if needed.

Install MongoDB tools and connect:

```
mongosh "mongodb://docdbadmin:PASSWORD@CLUSTER_ENDPOINT:27017/?replicaSet=rs0&readPreference=secondaryPreferred&retryWrites=false" --tls
```

Check databases:

```
show dbs
use flipkart-users
show collections
db.users.find()
```

LAB 13: Route 53 Custom Domain Optional

Step 13.1: Register or Use Domain

1. Open **Route 53**.
2. Register a domain or use an existing hosted zone.

Step 13.2: Create Hosted Zone

1. Go to **Hosted zones** → **Create hosted zone**.
2. Domain name: `yourdomain.com`.
3. Type: Public hosted zone.

Step 13.3: Create Alias Record

Setting	Value
Record name	blank for root or www
Record type	A
Alias	Yes
Route traffic to	Application Load Balancer
Region	us-east-1
Load balancer	flipkart-alb

LAB 14: ACM SSL Certificate Optional

Step 14.1: Request Certificate

1. Search for **Certificate Manager**.
2. Click **Request a certificate**.
3. Choose **Public certificate**.
4. Add domain names:
 - yourdomain.com
 - *.yourdomain.com
5. Validation method: DNS validation.
6. Request certificate.

Step 14.2: Validate Domain

1. Open certificate.
2. Click **Create records in Route 53**.
3. Wait until status becomes **Issued**.

Step 14.3: Add HTTPS Listener to ALB

1. Open flipkart-alb.
2. Go to **Listeners** → **Add listener**.
3. Configure:

Setting	Value
Protocol	HTTPS
Port	443
Default action	Forward to flipkart-frontend-tg
SSL certificate	Select ACM certificate

4. Add API path rule /api/* forwarding to flipkart-api-tg.

Step 14.4: Redirect HTTP to HTTPS

1. Edit HTTP:80 listener.
2. Default action: Redirect to HTTPS:443.
3. Status code: 301.

LAB 15: Cost Optimization

Step 15.1: Set Up Billing Alerts

1. Open **Billing Dashboard** → **Budgets**.
2. Click **Create budget**.
3. Configure:

Setting	Value
Budget type	Cost budget
Budget name	Flipkart-Monthly-Budget
Amount	\$100 or your limit
Alert threshold	80%
Email	Your email

Step 15.2: Enable Cost Explorer

1. Go to **Billing Dashboard** → **Cost Explorer**.
2. Click **Enable Cost Explorer**.
3. Wait up to 24 hours for data.

Step 15.3: Cost Saving Tips

- Use 1 task per service in development.
- Use 1 DocumentDB instance for development.
- Avoid NAT gateways in dev if possible.
- Stop non-production services during off hours.
- Set CloudWatch log retention to 7 days.
- Use GitHub Actions free minutes instead of Jenkins EC2 if suitable.

LAB 16: Backup and Disaster Recovery

Step 16.1: DocumentDB Automated Backups

1. Open **DocumentDB**.
2. Select `flipkart-docdb-cluster`.
3. Go to **Maintenance and backups**.
4. Confirm backup retention is 7 days.

Create manual snapshot:

1. Click **Actions** → **Take snapshot**.
2. Snapshot identifier: `flipkart-manual-backup-YYYY-MM-DD`.
3. Click **Create snapshot**.

Step 16.2: Export DocumentDB Data

Create bastion host:

Setting	Value
Name	DocumentDB-Bastion
AMI	Amazon Linux 2023 or Amazon Linux 2
Instance type	t3.micro
VPC	flipkart-vpc
Subnet	Public subnet
Public IP	Enabled
Security group	SSH from your IP only

Connect and export:

```
ssh -i key.pem ec2-user@BASTION_PUBLIC_IP
sudo yum install -y mongodb-mongosh mongodb-database-tools
mongodump --tls \
  --host CLUSTER_ENDPOINT:27017 \
  --username docdbadmin \
  --password PASSWORD \
  --out /tmp/backup
aws s3 cp /tmp/backup s3://flipkart-backups/$(date +%Y-%m-%d)/ --recursive
```

LAB 17: AWS Secrets Manager Setup

Step 17.1: Store DocumentDB Credentials

1. Search for **Secrets Manager**.
2. Click **Store a new secret**.
3. Secret type: Credentials for Amazon DocumentDB database.
4. Username: docdbadmin.
5. Password: DocumentDB password.
6. Encryption key: Default AWS managed key.
7. Select database: Your DocumentDB cluster.
8. Secret name: flipkart/docdb/credentials.
9. Store secret.

Step 17.2: Store JWT Secret

Secret type: Other type of secret.

JWT_SECRET=your-super-secret-jwt-key-change-this-to-something-secure-and-random

Secret name:

flipkart/jwt/secret

Step 17.3: Store Application Secrets

Secret name:

flipkart/app/config

Values:

```
{
  "EMAIL_USER": "your-email@gmail.com",
  "EMAIL_PASS": "your-app-password",
  "STRIPE_SECRET_KEY": "sk_test_XXXXX",
  "RAZORPAY_KEY_ID": "rzp_test_XXXXX",
  "RAZORPAY_KEY_SECRET": "XXXXX"
}
```

Step 17.4: Allow ECS to Read Secrets

1. Go to **IAM** → **Roles**.
2. Open ecsTaskExecutionRole.
3. Add inline policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
```

```
    "Action": [
      "secretsmanager:GetSecretValue",
      "kms:Decrypt"
    ],
    "Resource": [
      "arn:aws:secretsmanager:us-east-1:ACCOUNT_ID:secret:flipkart/*"
    ]
  }
}
```

Then create new task definition revisions and reference secrets under container **Secrets**.

LAB 18: ElastiCache Redis Setup

Step 18.1: Create Redis Subnet Group

1. Search for **ElastiCache**.
2. Go to **Subnet groups** → **Create subnet group**.
3. Configure:

Setting	Value
Name	flipkart-redis-subnet-group
VPC	flipkart-vpc
Subnets	Private subnets

Step 18.2: Create Redis Security Group

Name: flipkart-redis-sg

Inbound:

Type	Port	Source
Custom TCP	6379	flipkart-services-sg

Step 18.3: Create Redis Cluster

1. Go to **ElastiCache** → **Redis clusters**.
2. Click **Create Redis cluster**.
3. Configure:

Setting	Value
Cluster mode	Disabled
Name	flipkart-redis
Engine version	Redis 7.x
Port	6379
Node type	cache.t3.micro dev, cache.t3.small prod
Replicas	1
Subnet group	flipkart-redis-subnet-group
Security group	flipkart-redis-sg
Encryption at rest	Enabled
Encryption in transit	Enabled
Backups	Enabled, 7 days

Step 18.4: Save Redis Endpoint

Save primary endpoint:

flipkart-redis.xxxxx.cache.amazonaws.com:6379

Step 18.5: Application Redis Integration

Add redis dependency to product service:

```
{
  "dependencies": {
    "redis": "^4.6.0"
  }
}
```

Create redis-client.js:

```
const redis = require('redis');

const redisClient = redis.createClient({
  socket: {
    host: process.env.REDIS_HOST,
    port: process.env.REDIS_PORT || 6379,
    tls: true
  }
});

redisClient.on('error', (err) => console.log('Redis Client Error', err));
redisClient.on('connect', () => console.log('Redis Client Connected'));

async function connectRedis() {
  await redisClient.connect();
}

module.exports = { redisClient, connectRedis };
```

Cache product detail example:

```
const { redisClient, connectRedis } = require('./redis-client');
connectRedis().catch(console.error);

app.get('/:id', async (req, res) => {
  const productId = req.params.id;
  const cachedProduct = await redisClient.get(`product:${productId}`);
  if (cachedProduct) return res.json(JSON.parse(cachedProduct));

  const product = await Product.findById(productId);
  if (!product) return res.status(404).json({ error: 'Product not found' });

  await redisClient.setEx(`product:${productId}`, 3600, JSON.stringify(product));
  res.json(product);
});
```

Update ECS task definition:

```
REDIS_HOST=flipkart-redis.xxxxx.cache.amazonaws.com
REDIS_PORT=6379
```

LAB 19: CloudFront CDN Setup

Step 19.1: Create S3 Bucket for Static Assets

1. Search for **S3**.
2. Click **Create bucket**.
3. Configure:

Setting	Value
Bucket name	flipkart-static-assets-ACCOUNT_ID
Region	us-east-1
Versioning	Enabled
Encryption	SSE-S3
Public access	Keep blocked when using CloudFront OAC/OAI

Step 19.2: Upload Static Assets

Create folders:

```
/products/  
/banners/  
/assets/
```

Upload images, banners and static files.

Step 19.3: Create CloudFront Distribution

1. Search for **CloudFront**.
2. Click **Create distribution**.
3. Origin: ALB DNS name.
4. Protocol: HTTP only or HTTPS if ALB has certificate.
5. Default cache behavior:

Setting	Value
Compress objects	Yes
Viewer protocol policy	Redirect HTTP to HTTPS
Allowed methods	GET, HEAD, OPTIONS, PUT, POST, PATCH, DELETE
Cache policy	CachingOptimized for frontend
Origin request policy	AllViewer for dynamic apps
Default root object	index.html

Step 19.4: Add S3 Origin

Add S3 bucket as second origin using Origin Access Control or OAI.

Step 19.5: Create Behaviors

Path pattern	Origin	Cache policy
/static/*	S3 origin	CachingOptimized
/api/*	ALB origin	CachingDisabled
Default (*)	ALB origin	CachingOptimized or custom SPA policy

Step 19.6: Update Application URLs

If using CloudFront domain:

```
REACT_APP_API_URL=https://CLOUDFRONT_DOMAIN/api
REACT_APP_CDN_URL=https://CLOUDFRONT_DOMAIN/static
```

Rebuild and redeploy frontend image after changing React build-time variables.

LAB 20: AWS WAF Setup

Step 20.1: Create Web ACL

1. Search for **WAF & Shield**.
2. Click **Create web ACL**.
3. Configure:

Setting	Value
Name	flipkart-waf
CloudWatch metric name	flipkartWAF
Resource type	CloudFront distributions or Regional resources ALB
Associated resource	Select ALB or CloudFront

Step 20.2: Add Managed Rule Groups

Enable these AWS managed rules:

- Core rule set
- Known bad inputs
- SQL database
- Linux operating system

Optional paid rules:

- Bot Control
- Account takeover prevention

Step 20.3: Add Rate-Based Rule

Setting	Value
Name	RateLimitRule
Type	Rate-based rule
Rate limit	2000 requests per 5 minutes
IP address	Source IP
Action	Block

Step 20.4: Add Custom Rules

Examples:

- Block SQL injection in query strings.
- Block known high-risk geographies if your business allows it.
- Block suspicious URI patterns.

Step 20.5: Configure Metrics

Enable:

- CloudWatch metrics
- Sampled requests

Then create the Web ACL.

LAB 21: AWS Shield Advanced Optional

AWS Shield Standard is automatically enabled for:

- CloudFront
- Route 53
- Elastic Load Balancing

No action required for Shield Standard.

Shield Advanced is optional and costs about \$3,000/month. Use it only when you need enterprise-grade DDoS response, DRT access, and advanced protections.

LAB 22: AWS CodePipeline CI/CD

Step 22.1: Source Repository

Use either:

- GitHub
- AWS CodeCommit
- Bitbucket

For CodeCommit:

1. Search **CodeCommit**.
2. Create repository: `flipkart-microservices`.
3. Push your code.

Step 22.2: Create CodeBuild Project

1. Search for **CodeBuild**.
2. Click **Create build project**.
3. Example for API Gateway:

Setting	Value
Project name	flipkart-api-gateway-build
Source provider	GitHub or CodeCommit
Branch	main
OS	Amazon Linux 2
Runtime	Standard
Image	aws/codebuild/amazonlinux2-x86_64-standard:4.0
Privileged	Enabled required for Docker
Buildspec	api-gateway/buildspec.yml or service-specific path

Step 22.3: Example buildspec.yml

```
version: 0.2

phases:
  pre_build:
    commands:
      - echo Logging in to Amazon ECR...
      - aws ecr get-login-password --region $AWS_DEFAULT_REGION | docker login --username AWS
      --password-stdin $AWS_ACCOUNT_ID.dkr.ecr.$AWS_DEFAULT_REGION.amazonaws.com
      - REPOSITORY_URI=$AWS_ACCOUNT_ID.dkr.ecr.$AWS_DEFAULT_REGION.amazonaws.com/flipkart-api-
gateway
      - COMMIT_HASH=$(echo $CODEBUILD_RESOLVED_SOURCE_VERSION | cut -c 1-7)
      - IMAGE_TAG=${COMMIT_HASH:=latest}
  build:
    commands:
      - echo Build started on `date`
      - docker build -t $REPOSITORY_URI:latest ./api-gateway
      - docker tag $REPOSITORY_URI:latest $REPOSITORY_URI:$IMAGE_TAG
  post_build:
    commands:
```

```

- docker push $REPOSITORY_URI:latest
- docker push $REPOSITORY_URI:$IMAGE_TAG
- printf '["name":"api-gateway","imageUri":"%s"]' $REPOSITORY_URI:$IMAGE_TAG >
imagedefinitions.json

artifacts:
  files:
    - imagedefinitions.json

```

Step 22.4: CodeBuild IAM Permissions

Attach policies:

- AmazonEC2ContainerRegistryPowerUser
- ECS update permissions

Inline policy example:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ecs:DescribeTaskDefinition",
        "ecs:RegisterTaskDefinition",
        "ecs:UpdateService",
        "ecs:DescribeServices"
      ],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": ["iam:PassRole"],
      "Resource": "arn:aws:iam::*:role/ecsTaskExecutionRole"
    }
  ]
}

```

Step 22.5: Create CodePipeline

1. Search for **CodePipeline**.
2. Click **Create pipeline**.
3. Configure:

Stage	Configuration
Source	GitHub V2 or CodeCommit, branch main
Build	CodeBuild project
Deploy	Amazon ECS

Deploy settings:

Setting	Value
Cluster	flipkart-cluster
Service	api-gateway-service
Image definitions file	imagedefinitions.json

Repeat for all services or use a monorepo pipeline with multiple builds.

LAB 23: GitHub Actions CI/CD Alternative

Step 23.1: GitHub Secrets

Add repository secrets:

```
AWS_ACCESS_KEY_ID
AWS_SECRET_ACCESS_KEY
AWS_REGION=us-east-1
AWS_ACCOUNT_ID
```

Step 23.2: Example Workflow

Create `.github/workflows/deploy-api-gateway.yml`:

```
name: Deploy API Gateway

on:
  push:
    branches: [ main ]
    paths:
      - 'api-gateway/**'
  workflow_dispatch:

env:
  AWS_REGION: us-east-1
  ECR_REPOSITORY: flipkart-api-gateway
  ECS_SERVICE: api-gateway-service
  ECS_CLUSTER: flipkart-cluster
  ECS_TASK_DEFINITION: flipkart-api-gateway
  CONTAINER_NAME: api-gateway

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
          aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
          aws-region: ${ env.AWS_REGION }
      - id: login-ecr
        uses: aws-actions/amazon-ecr-login@v2
      - id: build-image
        env:
          ECR_REGISTRY: ${ steps.login-ecr.outputs.registry }
          IMAGE_TAG: ${ github.sha }
        run: |
          cd api-gateway
          docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG .
          docker tag $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG
          $ECR_REGISTRY/$ECR_REPOSITORY:latest
          docker push $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG
          docker push $ECR_REGISTRY/$ECR_REPOSITORY:latest
          echo "image=$ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG" >> $GITHUB_OUTPUT
      - run: |
          aws ecs describe-task-definition --task-definition ${ env.ECS_TASK_DEFINITION }
      - query taskDefinition > task-definition.json
      - id: task-def
        uses: aws-actions/amazon-ecs-render-task-definition@v1
        with:
          task-definition: task-definition.json
          container-name: ${ env.CONTAINER_NAME }
          image: ${ steps.build-image.outputs.image }
      - uses: aws-actions/amazon-ecs-deploy-task-definition@v1
        with:
          task-definition: ${ steps.task-def.outputs.task-definition }
          service: ${ env.ECS_SERVICE }
          cluster: ${ env.ECS_CLUSTER }
          wait-for-service-stability: true
```

Repeat with changed names for other services.

LAB 24: Jenkins on EC2 Alternative

Step 24.1: Launch Jenkins EC2 Instance

Setting	Value
Name	Jenkins-Server
AMI	Amazon Linux 2023 or Ubuntu 22.04
Instance type	t3.medium
VPC	flipkart-vpc
Subnet	Public subnet
Public IP	Enabled
Storage	30 GB gp3

Security group:

Port	Source
22	Your IP
8080	Your IP
443	Your IP

Step 24.2: Install Jenkins on Amazon Linux 2023

```

sudo yum update -y
sudo yum install java-17-amazon-corretto docker git unzip -y
sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.repo
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
sudo yum install jenkins -y
sudo systemctl start docker
sudo systemctl enable docker
sudo systemctl start jenkins
sudo systemctl enable jenkins
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install

```

Get initial admin password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Open:

```
http://JENKINS_PUBLIC_IP:8080
```

Step 24.3: Install Jenkins Plugins

Install:

- Docker Pipeline
- Amazon ECR
- Amazon ECS

- Pipeline: AWS Steps
- Git
- GitHub Integration
- CloudBees AWS Credentials

Step 24.4: Example Jenkinsfile

```

pipeline {
  agent any
  environment {
    AWS_REGION = 'us-east-1'
    AWS_ACCOUNT_ID = 'YOUR_ACCOUNT_ID'
    ECR_REPOSITORY = 'flipkart-api-gateway'
    ECS_CLUSTER = 'flipkart-cluster'
    ECS_SERVICE = 'api-gateway-service'
    IMAGE_TAG = "${env.BUILD_NUMBER}"
  }
  stages {
    stage('Checkout') {
      steps { checkout scm }
    }
    stage('Build Docker Image') {
      steps {
        dir('api-gateway') {
          sh 'docker build -t ${ECR_REPOSITORY}:${IMAGE_TAG} .'
        }
      }
    }
    stage('Push to ECR') {
      steps {
        withAWS(credentials: 'aws-credentials', region: "${AWS_REGION}") {
          sh '''
            aws ecr get-login-password --region ${AWS_REGION} | docker login --username AWS
            --password-stdin ${AWS_ACCOUNT_ID}.dkr.ecr.${AWS_REGION}.amazonaws.com
            docker tag ${ECR_REPOSITORY}:${IMAGE_TAG}
            ${AWS_ACCOUNT_ID}.dkr.ecr.${AWS_REGION}.amazonaws.com/${ECR_REPOSITORY}:${IMAGE_TAG}
            docker tag ${ECR_REPOSITORY}:${IMAGE_TAG}
            ${AWS_ACCOUNT_ID}.dkr.ecr.${AWS_REGION}.amazonaws.com/${ECR_REPOSITORY}:latest
            docker push
            ${AWS_ACCOUNT_ID}.dkr.ecr.${AWS_REGION}.amazonaws.com/${ECR_REPOSITORY}:${IMAGE_TAG}
            docker push
            ${AWS_ACCOUNT_ID}.dkr.ecr.${AWS_REGION}.amazonaws.com/${ECR_REPOSITORY}:latest
            '''
        }
      }
    }
    stage('Deploy to ECS') {
      steps {
        withAWS(credentials: 'aws-credentials', region: "${AWS_REGION}") {
          sh '''
            aws ecs update-service --cluster ${ECS_CLUSTER} --service ${ECS_SERVICE} --force-
            new-deployment --region ${AWS_REGION}
            '''
        }
      }
    }
  }
}

```

LAB 25: AWS X-Ray Tracing

Step 25.1: Create X-Ray IAM Role

1. Open **IAM** → **Roles** → **Create role**.
2. Trusted entity: AWS service.
3. Use case: Elastic Container Service Task.
4. Attach policy: AWSXRayDaemonWriteAccess.
5. Role name: ecsTaskRoleXRay.

Step 25.2: Add X-Ray Sidecar Container

In task definition revision:

Setting	Value
Container name	xray-daemon
Image	public.ecr.aws/xray/aws-xray-daemon:latest
Port	2000/udp
Essential	No
CPU	32 units
Memory	256 MB

Main container env var:

```
AWS_XRAY_DAEMON_ADDRESS=xray-daemon:2000
```

Task role:

```
ecsTaskRoleXRay
```

Step 25.3: Install X-Ray SDK

```
npm install aws-xray-sdk
```

Express example:

```
const AWSXRay = require('aws-xray-sdk');
const express = require('express');
const app = express();

app.use(AWSXRay.express.openSegment('APIGateway'));

// routes here

app.use(AWSXRay.express.closeSegment());
```

Step 25.4: View Traces

Open **X-Ray**:

- Service map

- Traces
- Analytics
- Error and latency insights

LAB 26: Enhanced Container Insights

Step 26.1: Enable Container Insights

If not already enabled:

1. Open **ECS** → **Clusters** → `flipkart-cluster`.
2. Click **Update cluster**.
3. Enable **Container Insights**.
4. Save.

Step 26.2: View Insights

Open **CloudWatch** → **Container Insights**:

- ECS Services performance
- CPU utilization
- Memory utilization
- Network usage
- Storage usage
- Map view

CloudWatch Logs Insights query:

```
fields @timestamp, @message
| filter @message like /ERROR/
| sort @timestamp desc
| limit 100
```

LAB 27: DataDog Integration Optional

Step 27.1: Create DataDog Account

1. Go to <https://www.datadoghq.com/>.
2. Start a trial.
3. Copy API key and application key.

Step 27.2: Store Keys in Secrets Manager

Secret name:

flipkart/datadog/keys

Values:

```
{
  "DD_API_KEY": "your-datadog-api-key",
  "DD_APP_KEY": "your-datadog-app-key",
  "DD_SITE": "datadoghq.com"
}
```

Step 27.3: Add DataDog Agent to ECS Task

Sidecar container:

Setting	Value
Container name	datadog-agent
Image	public.ecr.aws/datadog/agent:latest
Essential	No

Environment variables:

```
ECS_FARGATE=true
DD_APM_ENABLED=true
DD_APM_NON_LOCAL_TRAFFIC=true
```

Secrets:

DD_API_KEY from flipkart/datadog/keys

Main container env vars:

```
DD_AGENT_HOST=localhost
DD_TRACE_AGENT_PORT=8126
```

Install APM dependency:

```
npm install dd-trace
```

At top of server.js:

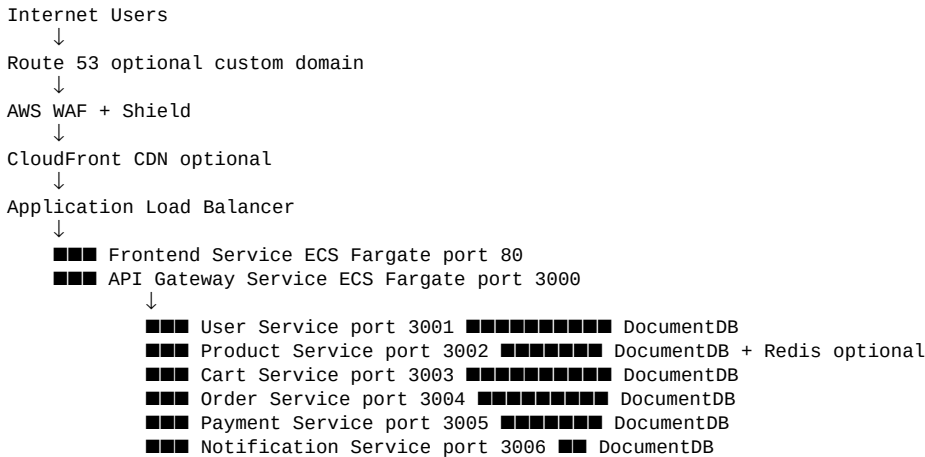
```
require('dd-trace').init({
  hostname: process.env.DD_AGENT_HOST,
```

```

    port: process.env.DD_TRACE_AGENT_PORT,
    service: 'api-gateway',
    env: 'production'
  });

```

Architecture Summary



Monitoring:
CloudWatch, CloudWatch Logs, Container Insights, X-Ray, optional DataDog

Security:
IAM, Security Groups, Secrets Manager, WAF, Shield, ACM TLS

CI/CD:
CodePipeline, GitHub Actions, or Jenkins

Complete Setup Checklist

Infrastructure

- ☐ VPC created with public and private subnets
- ☐ Internet Gateway configured
- ☐ NAT Gateway configured if using private outbound access
- ☐ Route tables verified
- ☐ Security groups configured

Container Registry

- ☐ 8 ECR repositories created
- ☐ All Docker images pushed to ECR
- ☐ Image scanning enabled

Database and Cache

- ☐ DocumentDB subnet group created

- ☐ DocumentDB cluster running
- ☐ DocumentDB security group allows ECS services
- ☐ Redis created if using cache
- ☐ Backups enabled

ECS

- ☐ ECS cluster created
- ☐ Task definitions created for all 8 services
- ☐ CloudWatch logging enabled
- ☐ ECS services running
- ☐ Cloud Map service discovery working

Load Balancing

- ☐ ALB created
- ☐ Frontend target group created
- ☐ API Gateway target group created
- ☐ Listener rule `/api/*` configured
- ☐ Targets healthy

Security

- ☐ Secrets stored in Secrets Manager
- ☐ ECS role can read secrets
- ☐ WAF configured
- ☐ ACM certificate issued if using HTTPS
- ☐ HTTP redirects to HTTPS

Monitoring

- ☐ CloudWatch dashboard created
- ☐ CloudWatch alarms created
- ☐ SNS alert email confirmed
- ☐ Container Insights enabled
- ☐ X-Ray enabled if required

Domain Optional

- ☐ Route 53 hosted zone created
- ☐ Alias record points to ALB or CloudFront
- ☐ SSL certificate attached

CI/CD Optional

- ☐ CodePipeline or GitHub Actions or Jenkins configured
- ☐ Build pushes image to ECR
- ☐ Deploy updates ECS service

Monthly Cost Estimate

Component	Specification	Approx Monthly Cost
ECS Fargate	8 services × 2 tasks × 0.5 vCPU, 1 GB	\$150
Jenkins EC2 optional	t3.medium	\$30
DocumentDB	db.t3.medium × 2	\$200
ElastiCache Redis	cache.t3.micro × 2	\$25
ALB	1 ALB + data transfer	\$20
NAT Gateway	2 NAT gateways + data	\$65
CloudFront	100 GB transfer	\$10
ECR	50 GB images	\$5
S3	100 GB static assets	\$3
DocumentDB backups	50 GB	\$5
WAF	Basic web ACL/rules	\$5
Secrets Manager	10 secrets	\$4
CloudWatch	Logs, metrics, alarms	\$10
X-Ray	1M traces	\$5
DataDog optional	APM Pro estimate	\$31
CodeBuild/CodePipeline	Basic usage	\$2
Total Full Setup		~\$537/month

Budget-friendly setup:

Optimization	Saving
Use 1 task per service	~\$75
Single DocumentDB instance	~\$100
Skip NAT gateways in dev	~\$65
Use GitHub Actions instead of Jenkins	~\$30
Skip DataDog	~\$31
Smaller Redis	~\$12

Reduced total estimate: ~\$224/month.

Performance Optimization Tips

Database Indexes

```
db.products.createIndex({ name: "text", description: "text" })
db.products.createIndex({ category: 1, price: 1 })
db.users.createIndex({ email: 1 }, { unique: true })
db.orders.createIndex({ userId: 1, createdAt: -1 })
```

Redis Caching Strategy

Product details: 1 hour TTL
Category listings: 30 minutes TTL
User sessions: 24 hours TTL
Cart data: 7 days TTL

CloudFront Cache-Control

Static assets: max-age=31536000
Images: max-age=604800
API responses: no-cache or CachingDisabled behavior

Auto Scaling

CPU target: 70%
Memory target: 80%
Min tasks: 2 production, 1 dev
Max tasks: 10

Troubleshooting Guide

Services Not Starting

Check:

1. ECS task logs.
2. ECR image URI and tag.
3. Environment variables.
4. Task execution role.
5. Secrets Manager permissions.
6. Security groups.
7. DocumentDB connection string.

Common fixes:

- Verify image exists in ECR.
- Recreate task definition revision.
- Confirm port mapping equals app port.
- Confirm health check endpoint exists.

DocumentDB Connection Failed

Check:

- DocumentDB SG allows port 27017 from services SG.
- ECS service is in same VPC/private subnet.
- Connection string includes `retryWrites=false`.
- Username and password are correct.
- TLS requirements for DocumentDB are satisfied.

ALB Health Checks Failing

Check:

- Target group port matches container port.
- Health check path is correct.
- Security group allows ALB to service.
- App listens on `0.0.0.0`, not only localhost.
- ECS tasks are healthy and logs show server startup.

Services Not Communicating

Check:

- Cloud Map namespace is private DNS.
- Service discovery names match environment variables.
- Security group allows internal service-to-service traffic.
- Use logs to confirm DNS resolution errors.

High Costs

Check:

- NAT Gateway hourly and data transfer charges.
- DocumentDB instance count and size.
- CloudWatch log ingestion volume.
- ECS desired task count.
- DataDog or paid WAF rule usage.

Final Next Steps

1. Deploy the application to ECS Fargate.
2. Validate each service with health checks.
3. Register user and test product/cart/order flow.
4. Add HTTPS with ACM.

5. Add WAF and CloudWatch alarms.
6. Add CI/CD using GitHub Actions or CodePipeline.
7. Move secrets to Secrets Manager.
8. Optimize cost after first test deployment.

Congratulations. Your Flipkart microservices application can now be deployed on AWS with a production-style architecture.

Application URL:

`http://YOUR_ALB_DNS_NAME`
`https://yourdomain.com`